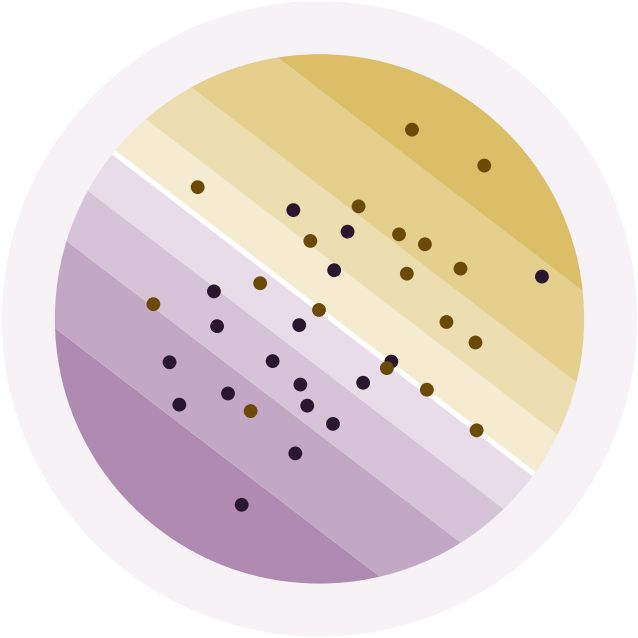
Логистическая регрессия и перцептрон

Урок



Четвёртый из пяти уроков о моделях — линейные классификаторы. Из урока «Ближайшие соседи» здесь нужны классы, доля верных ответов и стандартизация; из урока «Линейная регрессия» — свободный член как вес столбца единиц, градиент, градиентный спуск и штраф ridge. Пример — те же двенадцать пингвинов, сразу в стандартных единицах:

```
import numpy as np
# клюв, мм; масса, г; вид - 0, 1, 2
X = np.array([[38, 3600], [40, 3800], [36, 3300],
              [41, 3950], [47, 3550], [50, 3800],
              [46, 3400], [51, 3950], [46, 4900],
              [49, 5400], [41, 4650], [50, 5600]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2])
Z = (X - X.mean(axis=0)) / X.std(axis=0)
```

1 Логистическая регрессия

Для классов 0 и 1 линейная модель считает *отступ* $z = w \cdot x$ (свободный член внутри), а *сигмоида* переводит его в вероятность класса 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad p = \sigma(w \cdot x).$$

σ растёт от 0 до 1, $\sigma(0) = \frac{1}{2}$, $\sigma(-z) = 1 - \sigma(z)$. Класс 1 предсказывают при $p > \frac{1}{2}$, то есть при $w \cdot x > 0$: граница классов — гиперплоскость. Это *логистическая регрессия* — классификатор, хотя и зовётся регрессией. Веса подбирают по *логистической функции потерь*, log-loss:

$$L(w) = -\frac{1}{n} \sum_{i=1}^{n} \big(y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \big), \qquad p_i = \sigma(w \cdot x_i).$$

У объекта работает одно слагаемое — минус логарифм вероятности, которую модель дала верному ответу: уверенная верная модель платит почти 0, уверенная неверная — очень много. Если объекты независимы, nL — минус логарифм

вероятности всех ответов train, и минимум log-loss — веса, при которых наблюдённые ответы вероятнее всего.

Утверждение. $\nabla L(w) = \frac{1}{n} X^\top (p - y)$, где p — вектор вероятностей p_i.

Почему так. Производная сигмоиды — $\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z) \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big)$. Слагаемое объекта — $\ell = -y \ln p - (1-y)\ln(1-p)$ при $p = \sigma(z)$, и

$$\frac{d\ell}{dz} = -\frac{y}{p}\, p(1-p) + \frac{1-y}{1-p}\, p(1-p) = -y(1-p) + (1-y)p = p - y.$$

А $\partial z / \partial w_j = x_j$, так что $\partial \ell / \partial w_j = (p - y)\, x_j$, и сумма по объектам — $X^\top (p - y)$. Та же форма, что у MSE в уроке «Линейная регрессия»: «ошибка на признак», только ошибка — разность вероятности и ответа.

Уравнение $X^\top (p - y) = 0$ формулой не решается, и веса ищут спуском, $w \leftarrow w - \eta \cdot \frac{1}{n} X^\top (p - y)$. L выпукла, и при подходящем шаге спуск приходит к минимуму — если минимум есть. Пример: «папуанский или нет» по стандартизованной массе, свободный член 0, вес от 1 до 8:

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

t = (y == 2).astype(int)
A = np.column_stack([np.ones(12), Z[:, 1]])
for c in [1, 2, 4, 8]:
    p = sigmoid(A @ np.array([0, c]))
    loss = -(t * np.log(p) + (1 - t) * np.log(1 - p))
    print(c, loss.mean().round(4))
```

```
1 0.3734
2 0.2156
4 0.0876
8 0.0216
```

Ловушка. Чем больше вес, тем меньше потери, и так без конца: классы здесь разделимы — папуанские тяжелее всех, — и log-loss можно сделать сколь угодно малым, растягивая веса. Минимума нет, и спуск растит веса, пока его не остановят. Спасает штраф, как в ridge из урока «Линейная регрессия»; sklearn ставит его по умолчанию — раздел 3.

Ещё одна беда больших отступов — `np.exp` переполняется: `sigmoid(-800.0)` — это $1/(1+e^{800})$, а e^{800} в дробное число не помещается. Выходит бесконечность и предупреждение `RuntimeWarning: overflow encountered in exp`; σ при этом верна, 0, но $\ln 0 = -\infty$, и log-loss тоже бесконечен. Поэтому библиотеки считают log-loss формулами, которые обходят переполнение.

2 Перцептрон

Перцептрон — самый старый линейный классификатор. Классы — -1 и $+1$, ответ — знак $w \cdot x + b$. Обучение — проходы по объектам по порядку: если объект неверен или на границе, то есть $y_i(w \cdot x_i + b) \le 0$, веса поправляют, $w \leftarrow w + y_i x_i$ и $b \leftarrow b + y_i$; проходы повторяют, пока в проходе не будет ни одной поправки. Поправка сдвигает отступ ошибочного объекта в нужную сторону: новое $y_i(w \cdot x_i + b)$ больше старого на $\|x_i\|^2 + 1$.

Пример (перцептрон руками). $x_1 = (2, 0)$ класса $+1$, $x_2 = (0, 3)$ класса -1, $x_3 = (2, 2)$ класса $+1$; старт $w = (0, 0)$, $b = 0$. На x_1 отступ 0 — поправка: $w = (2, 0)$, $b = 1$. На x_2 $w \cdot x_2 + b = 1$ при классе -1 — поправка: $w = (2, -3)$, $b = 0$. На x_3 $4 - 6 = -2$ при классе $+1$ — поправка: $w = (4, -1)$, $b = 1$. Во втором проходе $w \cdot x_i + b$ равны 9, -2, 7, знаки верные, поправок нет; граница — прямая $4x_1 - x_2 + 1 = 0$. Если классы разделимы гиперплоскостью, перцептрон

останавливается за конечное число поправок (теорема Новикова); если нет, поправки не кончаются, и число проходов ограничивают.

3 То же в sklearn

Порядок вызовов — как в уроке «Ближайшие соседи»: создать объект с гиперпараметрами, `fit(X, y)`, `predict(X)`. `LogisticRegression`, как `Ridge` в уроке «Линейная регрессия», штрафует веса по умолчанию: минимизирует $\frac{1}{2}\|w\|^2 + C\sum_i \ell_i$ с $C = 1$ — сумма потерь, не среднее, свободный член без штрафа. Веса у неё меньше, чем у чистого log-loss, а на разделимых данных конечны; без штрафа — `C=np.inf`.

```
from sklearn.linear_model import LogisticRegression

zm = Z[:, [1]]
for C in [1, 100, np.inf]:
    lg = LogisticRegression(C=C).fit(zm, t)
    w, b = lg.coef_[0, 0], lg.intercept_[0]
    print(C, round(w, 2), round(b, 2))
```

```
1 1.62 -0.92
100 6.84 -1.74
inf 16.05 -3.31
```

Без штрафа вес на разделимых данных конечен лишь потому, что спуск внутри остановился по своему допуску. Вероятности классов — `predict_proba`, по столбцу на класс. Для K классов у модели по набору весов на класс, а вместо сигмоиды — softmax. В задаче, где модель задана точно — чистый log-loss, свой шаг и число шагов, — ответ sklearn с ней не совпадёт: у него штраф и свой способ спуска.

Перцептрон в sklearn тоже есть, `Perceptron`, но и он устроен иначе, чем алгоритм раздела 2: по умолчанию перемешивает объекты перед каждым проходом, `shuffle=True`, и

останавливается, когда потери несколько проходов подряд падают меньше чем на `tol`. Правило «по порядку, до прохода без поправок» — только своим кодом.

Дальше — урок «k-средних»: модель, которой ответы не нужны вовсе.